

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №5
По СПП

Выполнил:
студент 3 курса
группы ПО-12
Коханская Е.В.
Проверил:
Кулик А. Д.

```
"""FastAPI application for rental service."""
```

```
from typing import List, Optional
```

```
from fastapi import FastAPI, Depends, HTTPException
```

```
from sqlalchemy.orm import Session
```

```
import models
```

```
import schemas
```

```
from database import engine, get_db
```

```
from crud import CustomerCRUD, ProductCategoryCRUD, ProductCRUD, RentalCRUD, PaymentCRUD
```

```
models.Base.metadata.create_all(bind=engine)
```

```
app = FastAPI(title="Rental Service API", version="1.0.0")
```

```
@app.get("/")
```

```
def root():
```

```
    """Root endpoint."""
```

```
    return {"message": "Welcome to Rental Service API", "version": "1.0.0"}
```

```
# Customer endpoints
```

```
@app.post("/customers/", response_model=schemas.CustomerResponse)
```

```
def create_customer(customer: schemas.CustomerCreate, db: Session = Depends(get_db)):
```

```
    """Create new customer."""
```

```
    return CustomerCRUD.create(db, customer)
```

```
@app.get("/customers/", response_model=List[schemas.CustomerResponse])
```

```
def get_customers(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
```

```
    """Get all customers."""
```

```
    return CustomerCRUD.get_all(db, skip, limit)
```

```
@app.get("/customers/{customer_id}", response_model=schemas.CustomerResponse)
```

```
def get_customer(customer_id: int, db: Session = Depends(get_db)):
```

```

"""Get customer by ID."""
customer = CustomerCRUD.get_by_id(db, customer_id)

if not customer:
    raise HTTPException(status_code=404, detail="Customer not found")

return customer


@app.put("/customers/{customer_id}", response_model=schemas.CustomerResponse)
def update_customer(
    customer_id: int, customer: schemas.CustomerCreate, db: Session = Depends(get_db)
):
    """Update customer."""
    updated = CustomerCRUD.update(db, customer_id, customer)

    if not updated:
        raise HTTPException(status_code=404, detail="Customer not found")

    return updated


@app.delete("/customers/{customer_id}")
def delete_customer(customer_id: int, db: Session = Depends(get_db)):
    """Delete customer."""
    success = CustomerCRUD.delete(db, customer_id)

    if not success:
        raise HTTPException(status_code=404, detail="Customer not found")

    return {"message": "Customer deleted successfully"}


# Product Category endpoints
@app.post("/categories/", response_model=schemas.ProductCategoryResponse)
def create_category(
    category: schemas.ProductCategoryCreate, db: Session = Depends(get_db)
):
    """Create new product category."""
    return ProductCategoryCRUD.create(db, category)


@app.get("/categories/", response_model=List[schemas.ProductCategoryResponse])
def get_categories(db: Session = Depends(get_db)):

```

```

        """Get all product categories."""
        return ProductCategoryCRUD.get_all(db)

# Product endpoints

@app.post("/products/", response_model=schemas.ProductResponse)
def create_product(product: schemas.ProductCreate, db: Session = Depends(get_db)):
    """Create new product."""
    return ProductCRUD.create(db, product)

@app.get("/products/", response_model=List[schemas.ProductResponse])
def get_products(
    skip: int = 0,
    limit: int = 100,
    available_only: bool = False,
    db: Session = Depends(get_db),
):
    """Get all products."""
    return ProductCRUD.get_all(db, skip, limit, available_only)

@app.get("/products/{product_id}", response_model=schemas.ProductResponse)
def get_product(product_id: int, db: Session = Depends(get_db)):
    """Get product by ID."""
    product = ProductCRUD.get_by_id(db, product_id)
    if not product:
        raise HTTPException(status_code=404, detail="Product not found")
    return product

@app.put("/products/{product_id}", response_model=schemas.ProductResponse)
def update_product(
    product_id: int, product: schemas.ProductCreate, db: Session = Depends(get_db)
):
    """Update product."""
    updated = ProductCRUD.update(db, product_id, product)
    if not updated:

```

```

        raise HTTPException(status_code=404, detail="Product not found")

    return updated

@app.delete("/products/{product_id}")
def delete_product(product_id: int, db: Session = Depends(get_db)):
    """Delete product."""

    success = ProductCRUD.delete(db, product_id)

    if not success:
        raise HTTPException(status_code=404, detail="Product not found")

    return {"message": "Product deleted successfully"}

# Rental endpoints

@app.post("/rentals/", response_model=schemas.RentalResponse)
def create_rental(rental: schemas.RentalCreate, db: Session = Depends(get_db)):
    """Create new rental."""

    new_rental = RentalCRUD.create(db, rental)

    if not new_rental:
        raise HTTPException(status_code=400, detail="Product not available for rent")

    return new_rental

@app.get("/rentals/", response_model=List[schemas.RentalResponse])
def get_rentals(
    skip: int = 0,
    limit: int = 100,
    customer_id: Optional[int] = None,
    db: Session = Depends(get_db),
):
    """Get all rentals."""

    return RentalCRUD.get_all(db, skip, limit, customer_id)

@app.post("/rentals/{rental_id}/return", response_model=schemas.RentalResponse)
def return_rental(rental_id: int, db: Session = Depends(get_db)):
    """Mark rental as returned."""

    returned = RentalCRUD.return_product(db, rental_id)

```

if not returned:

```
    raise HTTPException(
        status_code=404, detail="Rental not found or already returned"
    )
```

return returned

Payment endpoints

```
@app.post("/payments/", response_model=schemas.PaymentResponse)
```

```
def create_payment(payment: schemas.PaymentCreate, db: Session = Depends(get_db)):
```

```
    """Create new payment."""
```

```
    return PaymentCRUD.create(db, payment)
```

```
@app.get(
```

```
    "/payments/customer/{customer_id}", response_model=List[schemas.PaymentResponse]
```

```
)
```

```
def get_customer_payments(
```

```
    customer_id: int, skip: int = 0, limit: int = 100, db: Session = Depends(get_db)
```

```
):
```

```
    """Get payments by customer."""
```

```
    return PaymentCRUD.get_by_customer(db, customer_id, skip, limit)
```